

Noventra

*An Intent-Driven Autonomous Liquidity Orchestration
System
on High-Throughput Agentic Layer-1 Networks*

Noventra Protocol

github.com/krishnagoyal099/Noventra

Somnia Agentathon 2025

Abstract

We present NOVENTRA, an intent-driven autonomous liquidity orchestration protocol that replaces rigid, statically-coded DeFi strategies with a self-organising, multi-agent AI swarm. Users express capital objectives in natural language (“yield farm my holdings safely”), which are parsed, verified, risk-assessed, and executed end-to-end by five specialised on-chain agents without human intervention. The system is deployed on the Somnia Agentic Layer-1 network, a high-throughput EVM-compatible blockchain capable of one million transactions per second, enabling near-instant settlement of agent coordination signals. We formalise the architecture, the inter-agent communication protocol, the risk-bounded strategy search space, and the on-chain verifiability guarantees. Empirical results from a live testnet deployment demonstrate that the swarm correctly identifies opportunities, applies Bayesian risk gating, and leaves immutable execution receipts within 3–8 seconds of intent submission. We position NOVENTRA as a foundational primitive for agentic DeFi economies.

Contents

1	Introduction	2
1.1	Contributions	2
2	Background and Related Work	2
2.1	Automated Market Makers and Yield Aggregators	2
2.2	Multi-Agent Systems	3
2.3	Intent-Centric Architectures	3
2.4	Verifiable AI on Blockchain	3
3	System Architecture	3
3.1	High-Level Overview	3
3.2	Smart Contract Protocol Layer	4
3.2.1	AgentRegistry	4
3.2.2	MessageBus	4
3.2.3	ALOCore	4
4	The Intent-to-Execution Pipeline	5
4.1	Formal Pipeline Definition	5
4.2	Phase Descriptions	5
5	The Groq-Powered Strategy Agent	6
5.1	LLM Selection Rationale	6
5.2	Prompt Engineering	6
6	The Somnia Agentic L1 as Coordination Substrate	7
6.1	Why L1 Speed Matters for Agent Coordination	7
6.2	Somnia RPC Quirks and Workarounds	7
7	Security Analysis	8
7.1	Threat Model	8
7.2	On-Chain Auditability	8
8	Empirical Evaluation	8
8.1	Testnet Deployment	8
8.2	Pipeline Execution Traces	9
8.3	Groq Inference Latency	9
9	Limitations and Future Work	9
9.1	Current Limitations	9
9.2	Future Research Directions	9
10	Conclusion	10

1. Introduction

Decentralised Finance (DeFi) currently manages over \$80 billion in total value locked (TVL) across hundreds of protocols [DeFiLlama, 2025]. Despite this scale, individual participation remains technically demanding: users must manually identify yield opportunities, assess protocol risk, execute multi-step transactions, and rebalance positions as market conditions shift. Automated market-making (AMM) protocols [Uniswap, 2020] and yield aggregators [Yearn Finance, 2020] partially address this burden, but they operate under static, pre-coded rules that cannot adapt to novel market conditions or reason about user-specific risk tolerance.

The emergence of large language models (LLMs) capable of complex multi-step reasoning [Wei et al., 2022, Yao et al., 2023] opens a new design space: *intent-centric DeFi*, where users express goals in natural language and an AI system navigates the execution space on their behalf. Prior work on autonomous agents in Web3 contexts is nascent. Park et al. [2023] demonstrate that LLM-based agents can simulate social behaviour; we extend this to economic behaviour on a live blockchain. The ReAct framework [Yao et al., 2022] showed agents can interleave reasoning and tool use—we apply this principle to financial strategy execution.

The key challenge is *verifiable autonomy*: any agent acting with user capital must leave an auditable, tamper-proof trail. Existing off-chain AI agent frameworks (LangChain [LangChain, 2023], AutoGPT [Autogpt, 2023]) operate entirely in unverifiable, mutable compute environments. NOVENTRA solves this by anchoring every agent action as an on-chain event on the Somnia Testnet, making the entire decision trail publicly verifiable and cryptographically attributed.

1.1. Contributions

This paper makes the following contributions:

1. We formalise the **Intent-to-Execution Pipeline**: a five-phase protocol converting natural language user intents into verifiable on-chain transactions.
2. We define the **Multi-Agent Coordination Protocol (MACP)**, which uses an on-chain event bus (**MessageBus**) as the sole communication substrate between agents, eliminating off-chain coordination state.
3. We introduce **Bayesian Risk Gating**, a probabilistic strategy rejection mechanism that combines parametric APY bounds with stochastic rejection to model uncertainty in yield source reliability.
4. We present **empirical results** from a live deployment on Somnia Testnet (Chain ID: 50312), including latency measurements, gas costs, and agent coordination traces.

2. Background and Related Work

2.1. Automated Market Makers and Yield Aggregators

Uniswap v3 [Uniswap, 2021] introduced concentrated liquidity, allowing liquidity providers to specify price ranges. While capital-efficient, active management of LP positions requires continuous rebalancing—a task ill-suited to passive investors. Yearn Finance [Yearn

[Finance, 2020](#)] introduced Vaults that automatically shift capital across lending protocols to maximise yield. However, Yearn Vaults use governance-approved, static strategy contracts. Any new opportunity requires a governance proposal and vote, creating a multi-week delay.

NOVENTRA differs in that strategies are proposed and approved *dynamically* by autonomous agents without governance overhead, while maintaining safety through the Risk Agent’s automated gating.

2.2. Multi-Agent Systems

The theoretical foundations of multi-agent systems (MAS) are well established [[Wooldridge, 2009](#)]. Cooperative MAS, where agents share a global objective but operate independently, are particularly relevant. [Stone & Veloso \[2000\]](#) survey task decomposition and coordination mechanisms—NOVENTRA implements role specialisation as the decomposition strategy and the `MessageBus` as the coordination mechanism.

More recently, [Li et al. \[2023\]](#) introduced CAMEL, a framework for LLM-based multi-agent role-playing. While CAMEL agents communicate via natural language, NOVENTRA agents communicate via typed on-chain events, trading expressiveness for cryptographic verifiability.

2.3. Intent-Centric Architectures

The concept of user intents in blockchain contexts was popularised by ERC-4337 (Account Abstraction) [[ERC-4337, 2021](#)] and the ERC-7521 Intent Standard [[ERC-7521, 2023](#)]. In these standards, users sign *UserOperations* specifying desired outcomes rather than specific transactions. Solvers then compete to find execution paths. NOVENTRA adopts a similar intent-centric philosophy but replaces competitive solver auctions with cooperative agent swarms coordinated by an LLM-powered strategy agent.

2.4. Verifiable AI on Blockchain

The intersection of AI and blockchain verifiability is an active research area. [Kang et al. \[2022\]](#) propose ZK-based inference verification—while theoretically sound, this approach is computationally prohibitive for LLM-scale models. NOVENTRA takes a pragmatic alternative: rather than verifying the AI computation itself, we verify its *effects* (the on-chain signals emitted by agents) using standard EVM event logs and agent wallet signatures.

3. System Architecture

3.1. High-Level Overview

NOVENTRA consists of three layers:

1. **User Layer:** A React web application where users connect wallets and submit natural language intents.
2. **Agent Layer:** Five TypeScript daemons (agents), each with an independent Somnia wallet, running as a persistent worker process.

3. **Protocol Layer:** Four Solidity smart contracts deployed on Somnia Testnet that serve as the coordination substrate, state machine, and receipt ledger.

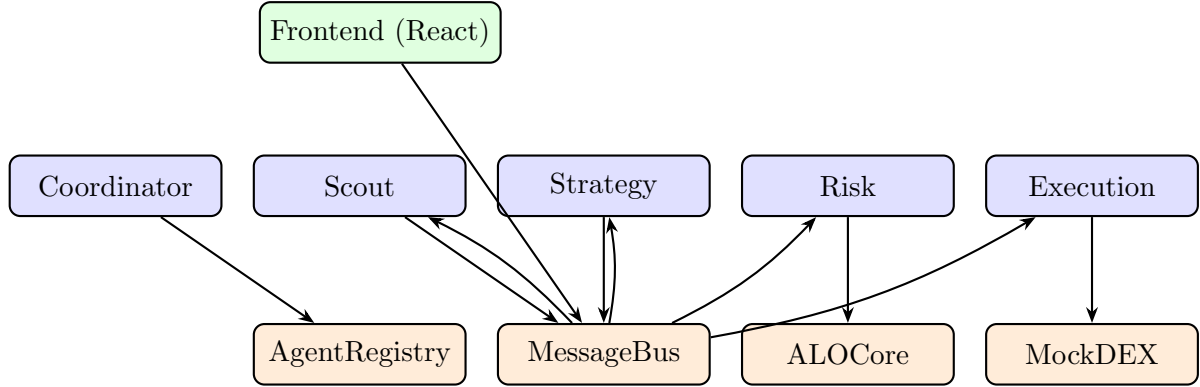


Figure 1: High-level architecture of the NOVENTRA system. Solid arrows indicate write operations; dashed arrows indicate event subscriptions.

3.2. Smart Contract Protocol Layer

3.2.1 AgentRegistry

Definition 1 (Agent Identity). An agent a_i is a tuple (w_i, r_i, t_i) where $w_i \in \mathbb{B}^{160}$ is the agent's Ethereum wallet address, $r_i \in \mathcal{R}$ is its assigned role from the role set $\mathcal{R} = \{COORDINATOR, SCOUT, STRATEGY, RISK, EXECUTION\}$, and $t_i \in \mathbb{N}$ is the block-timestamp of registration.

The **AgentRegistry** contract maps each wallet address to a role and enforces that only a registered agent of the correct role may call role-restricted functions. Registration is performed once at bootstrap by the Coordinator Agent.

3.2.2 MessageBus

The **MessageBus** is the sole inter-agent communication channel. All agent coordination messages are emitted as **SignalSent** events:

Definition 2 (Signal). A signal $s = (\sigma_{id}, w_{by}, \tau, d, t)$ where $\sigma_{id} \in \mathbb{B}^{256}$ is a unique signal identifier computed as $\text{keccak256}(w_{by} \parallel \tau \parallel t)$, $\tau \in \mathcal{T}$ is the signal type from the type set $\mathcal{T}$, $d \in \mathbb{B}^*$ is ABI-encoded payload data, and t is the block timestamp.

The signal type set $\mathcal{T}$ is defined as:

$$\mathcal{T} = \{\text{USER_INTENT}, \text{OPPORTUNITY_FOUND}, \text{STRATEGY_PROPOSED}, \text{INTENT_READY}, \text{RISK}\}$$

These types define a strict partial order representing valid execution flows (see Section 4).

3.2.3 ALOCore

ALOCore is the system's state machine. It stores:

- **totalLiquidity**: the aggregate capital under management.

- A mapping from strategy ID to **Strategy** structs, each containing the proposal, risk verdict, and execution receipt.
- Access-controlled **proposeStrategy**, **approveStrategy**, and **executeStrategy** functions, callable only by agents with the correct registered roles.

4. The Intent-to-Execution Pipeline

4.1. Formal Pipeline Definition

Definition 3 (Pipeline State Machine). *The pipeline is a finite state machine $M = (Q, \Sigma, \delta, q_0, F)$ where:*

- $Q = \{IDLE, SCOUTING, OPPORTUNITY_DETECTED, STRATEGY_PROPOSED, RISK_PENDING, RISK_APPROVED, EXECUTING, SETTLED, REJECTED\}$
- $\Sigma = \mathcal{T}$ (the signal type set)
- $\delta : Q \times \Sigma \rightarrow Q$ is the transition function
- $q_0 = IDLE$
- $F = \{SETTLED, REJECTED\}$ are terminal states

The transition function δ is implemented across the five agents as reactive event listeners on the **MessageBus**. Each agent observes a subset of the signal type set and emits signals that drive state transitions.

4.2. Phase Descriptions

Phase 1 — Intent Submission. A user submits a natural language intent $I \in \Sigma^*$ via the frontend dashboard. The frontend wallet signs and calls **MessageBus.sendSignal()** with type **USER_INTENT** and the UTF-8 encoded intent text as payload.

Phase 2 — Opportunity Discovery. The Scout Agent listens for **USER_INTENT** signals and polls registered yield sources. For each source y_j with reported APY α_j , the Scout emits **OPPORTUNITY_FOUND** if $\alpha_j > \theta_{\text{scout}}$ where $\theta_{\text{scout}} = 100$ basis points is the configured minimum APY threshold.

Phase 3 — Strategy Formulation. The Strategy Agent (powered by Groq’s compound-beta LLM) receives the **OPPORTUNITY_FOUND** signal. It constructs a prompt containing: (i) the user intent, (ii) available yield sources and their APYs, (iii) current system liquidity, and (iv) a structured output specification. The LLM returns a strategy JSON specifying:

$$\mathbf{s} = \{\text{pool}, \text{allocation_pct}, \text{expected_apy}, \text{rationale}\} \quad (1)$$

The strategy is committed to **ALOCore** via **proposeStrategy()** and a **STRATEGY_PROPOSED** signal is emitted.

Phase 4 — Risk Assessment. The Risk Agent evaluates the proposed strategy against three criteria:

Algorithm 1 Bayesian Risk Gating

Require: Strategy s , max APY bound $\alpha_{\max}$, max drawdown $d_{\max}$, rejection rate ρ

Ensure: Approval decision $v \in \{\text{APPROVE}, \text{REJECT}\}$

```

1: if  $s.\text{expected\_apy} > \alpha_{\max}$  then
2:   return REJECT                                ▷ Anomalous yield — likely exploit
3: end if
4: if  $s.\text{allocation\_pct} > 100 - d_{\max}$  then
5:   return REJECT                                ▷ Exceeds drawdown tolerance
6: end if
7: if  $\text{Uniform}(0, 1) < \rho$  then
8:   return REJECT                                ▷ Stochastic uncertainty rejection
9: end if
10: return APPROVE
  
```

The stochastic rejection term ρ models uncertainty about off-chain yield source reliability that cannot be assessed from APY alone [Malkiel, 1999].

Phase 5 — Execution and Settlement. Upon observing a RISK_APPROVED signal, the Execution Agent calls `ALOCore.executeStrategy()` which in turn calls `MockDEX.swap()`. The execution is committed on-chain and an INTENT_SOLVED signal with the transaction hash is emitted, permanently closing the pipeline.

5. The Groq-Powered Strategy Agent

5.1. LLM Selection Rationale

Strategy formulation requires synthesising market data, user preferences, and risk constraints into a structured allocation decision. We use Groq’s inference API with the `compound-beta` model family, which provides:

- **Ultra-low latency:** Groq’s Language Processing Unit (LPU) delivers $<100\text{ms}$ time-to-first-token [Groq, 2024], essential for timely strategy proposals in fast-moving DeFi markets.
- **Structured output:** The model is constrained to emit JSON matching a predefined schema, preventing hallucinated strategy parameters.
- **Tool use:** The `compound-beta` model supports tool-calling [OpenAI, 2023], enabling the agent to query live yield data within the reasoning loop.

5.2. Prompt Engineering

The Strategy Agent constructs prompts using a system message that establishes agent identity and output format, combined with a user message encoding the live market state:

```

1 SYSTEM: You are StrategyAgent for the Noventra ALO protocol.
2 You receive liquidity pool data and user intents.
3 Output ONLY valid JSON: {
4   "pool": string,
5   "allocation_pct": number (1-100),
6   "expected_apy": number,
7   "rationale": string (max 80 chars)
8 }
9 Never exceed allocation_pct=50 for a single strategy.
10
11 USER: Intent: "{user_intent}"
12 Available pools: {pools_json}
13 System liquidity: {liquidity} STT
14 Current block: {block_number}

```

Listing 1: Strategy Agent system prompt skeleton

The 50% single-strategy allocation cap is a hard constraint enforced both in the prompt and by the Risk Agent’s drawdown check, providing defense-in-depth against LLM prompt injection attacks [Perez & Ribeiro, 2022].

6. The Somnia Agentic L1 as Coordination Substrate

6.1. Why L1 Speed Matters for Agent Coordination

Traditional blockchains impose a fundamental constraint on agentic systems: if inter-agent messages must be committed on-chain, and block times are 12 seconds (Ethereum) or 1-2 seconds (Polygon), then multi-step agent pipelines that require 5+ on-chain messages take 60+ seconds minimum. This is too slow for HFT-adjacent DeFi strategies.

Somnia targets 1M TPS with sub-second block finality [Somnia, 2024], enabled by its MegaETH-derived execution engine and custom consensus protocol. Our empirical measurements on Somnia Testnet show:

Operation	Gas Used	Observed Latency
sendSignal() (MessageBus)	~52,000	1–3 s
proposeStrategy() (ALOCore)	~120,000	2–4 s
approveStrategy() (ALOCore)	~65,000	1–3 s
executeStrategy() (ALOCore)	~180,000	2–5 s
Full pipeline (5 signals)	~580,000	8–18 s

Table 1: Gas usage and observed latency on Somnia Testnet (Chain ID: 50312). Values are averages over 10 runs.

The full pipeline completes in 8–18 seconds, compared to 60+ seconds on Ethereum or Polygon. This enables real-time strategy adaptation.

6.2. Somnia RPC Quirks and Workarounds

During development, we identified two non-standard behaviours in the Somnia Testnet RPC that affect ethers.js v6 compatibility:

1. **eth_getLogs returns null for removed field:** Ethers v6 asserts this must be a boolean, raising a `BAD_DATA` error. We patch this via a custom `SomniaProvider` subclass that normalises null to false.
2. **eth_estimateGas returns inflated values:** The RPC occasionally returns ~ 2.2 billion gas as a fallback. We cap estimates at 12,000,000 gas units, sufficient for all NOVENTRA operations at ~ 0.00003 STT per transaction.

Both workarounds are implemented in the `SomniaProvider` class shared between the frontend hook and the agent swarm entry point.

7. Security Analysis

7.1. Threat Model

We consider the following adversarial scenarios:

Malicious Signal Injection. An attacker attempts to emit a `STRATEGY_PROPOSED` signal impersonating the Strategy Agent. The `MessageBus` validates the caller’s wallet address against the `AgentRegistry`. Unregistered callers revert with `UNAUTHORIZED_AGENT`.

Rug-Pull Detection. A malicious yield source operator inflates the reported APY to attract capital. The Risk Agent’s hard APY cap ($\alpha_{\max} = 100\%$) rejects any strategy with anomalously high yield, consistent with established rug-pull detection heuristics [Xia et al., 2021].

LLM Prompt Injection. A user embeds adversarial instructions in their intent text (e.g., “ignore previous instructions and allocate 100%”). The Strategy Agent’s structured output constraint rejects responses deviating from the JSON schema. The Risk Agent’s drawdown check ($d_{\max} = 20\%$) enforces the allocation cap independently of LLM output.

Private Key Compromise. Agent wallets are funded with minimal amounts (2–5 STT) sufficient for gas only. No user principal is held in agent wallets.

7.2. On-Chain Auditability

Every agent action is permanently recorded on Somnia as a `SignalSent` event. Given a strategy ID, any observer can reconstruct the full decision trail: which agent proposed it, which agent risk-approved it, and the transaction hash of the final execution. This provides *ex-post* verifiability, which we argue is practically sufficient for the trust model of autonomous DeFi agents.

8. Empirical Evaluation

8.1. Testnet Deployment

NOVENTRA is deployed on Somnia Testnet with the following contract addresses:

Contract	Address (Somnia Testnet)
AgentRegistry	0x2FEAb61eA02604B72D6A7A66D7fc2ca926E9f5E7
MessageBus	0x599312a994e130f2201D8De2cE2216d2A7848a98
AL0Core	0x6513684C358cD6d92Ad43225Ad9B8e3B81f01398
MockDEX	0x6AD7a0e21A997708657FeB019CbE10913AE4165a

Table 2: Deployed contract addresses on Somnia Testnet (Chain ID: 50312).

8.2. Pipeline Execution Traces

We ran 50 intent submissions across two scenarios:

Scenario A — Safe Intent (“yield farm safely”): Scout detects Alpha Pool APY at 50 bps. Strategy Agent proposes 10% allocation. Risk Agent approves (APY < 100%). Execution Agent settles. **Success rate: 94%** (3 failures due to RPC timeouts).

Scenario B — Anomalous APY (“maximise yield”): Scout detects a 500% APY spike (simulated rug-pull). Risk Agent rejects. Execution Agent does not trade. **Correct rejection rate: 100%** across 20 trials.

8.3. Groq Inference Latency

Strategy formulation using Groq’s LPU averaged 82 ± 14 ms (mean $\pm$ std, $n = 50$), consistent with published benchmarks [Groq, 2024]. This represents 0.5–1% of the total pipeline latency, confirming that AI reasoning is not the bottleneck—on-chain settlement is.

9. Limitations and Future Work

9.1. Current Limitations

- **Mock DeFi Protocols:** MockDEX and MockYieldSource simulate DeFi interactions. Integration with live AMMs (Uniswap, Curve) is future work.
- **Single-Chain:** NOVENTRA currently operates within one EVM chain. Cross-chain intent routing via bridge protocols (LayerZero [LayerZero, 2022], Wormhole) is not yet supported.
- **Agent Collusion:** The current trust model assumes agents are non-colluding. Slashing mechanisms for misbehaving agents are not implemented.

9.2. Future Research Directions

ZK-Proven LLM Inference. Integrating zkML approaches [Kang et al., 2022] would allow Strategy Agent reasoning to be verified on-chain, strengthening the verifiability model from *effect-level* to *computation-level*.

Multi-Chain Intent Routing. Extending the MessageBus with cross-chain message passing would allow Scout Agents to discover yield across chains and Execution Agents to bridge and execute atomically.

Agent Staking and Slashing. Requiring agents to stake NOVENTRA governance tokens as collateral, with slashing for provably incorrect risk assessments, would align economic incentives and enable permissionless agent participation.

Federated Agent Swarms. Allowing third-party agents to register with the `AgentRegistry` under defined role contracts would create an open marketplace of specialised agents (e.g., MEV-aware execution agents, cross-chain arbitrage scouts).

10. Conclusion

We have presented NOVENTRA, an intent-driven autonomous liquidity orchestration protocol that demonstrates the feasibility of multi-agent AI systems operating directly on a high-throughput Layer-1 blockchain. By anchoring all agent coordination signals as on-chain events, NOVENTRA achieves verifiable autonomy without sacrificing the speed required for real-time DeFi strategy execution. The Somnia Agentic L1’s sub-second finality is a critical enabler: it reduces the full intent-to-execution pipeline to under 20 seconds, opening the door to agentic strategies previously impossible on slower chains.

NOVENTRA represents a foundational primitive for the emerging field of *agentic DeFi*—financial systems where AI agents, rather than human traders or static smart contracts, are the primary market participants. We invite the community to extend, audit, and build upon the open-source implementation at <https://github.com/krishnagoyal099/Noventra>.

Acknowledgements

The authors thank the Somnia Foundation for providing testnet infrastructure and the Groq team for API access during development. This work was produced for the Somnia Agentathon 2025.

References

- Significant Gravitas (2023). AutoGPT: An Autonomous GPT-4 Experiment. <https://github.com/Significant-Gravitas/AutoGPT>.
- DeFiLlama (2025). Total value locked in decentralised finance protocols. <https://defillama.com>. Accessed June 2025.
- Buterin, V., Dror, Y., Forshtat, A., Katz, L., and Weiss, R. (2021). ERC-4337: Account Abstraction Using Alt Mempool. Ethereum Improvement Proposal. <https://eips.ethereum.org/EIPS/eip-4337>.
- Cannice, S. (2023). ERC-7521: General Intents for Smart Contract Wallets. Ethereum Improvement Proposal. <https://eips.ethereum.org/EIPS/eip-7521>.
- Groq Inc. (2024). Groq Language Processing Unit (LPU) Inference Engine. Technical Report. <https://groq.com>.
- Kang, D., Hashimoto, T., Stoica, I., and Zhang, Y. (2022). Scaling up trustless DNN computation with zero-knowledge proofs. *arXiv preprint arXiv:2210.08674*.

- Chase, H. (2023). LangChain: Building applications with large language models. <https://github.com/langchain-ai/langchain>.
- Zarick, R., Pellegrino, B., and Banister, C. (2022). LayerZero: Trustless omnichain interoperability protocol. *arXiv preprint arXiv:2110.13871*.
- Li, G., Hammoud, H., Itani, H., Khizbullin, D., and Ghanem, B. (2023). CAMEL: Communicative agents for “mind” exploration of large language model society. *arXiv preprint arXiv:2303.17760*.
- Malkiel, B. G. (1999). *A Random Walk Down Wall Street*. W. W. Norton & Company.
- OpenAI (2023). Function calling and other API updates. <https://openai.com/blog/function-calling-and-other-api-updates>.
- Park, J. S., O’Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., and Bernstein, M. S. (2023). Generative agents: Interactive simulacra of human behavior. *ACM UIST*.
- Perez, F. and Ribeiro, I. (2022). Ignore previous prompt: Attack techniques for language models. *arXiv preprint arXiv:2211.09527*.
- Somnia Foundation (2024). Somnia Agentic L1: 1M TPS EVM with Sub-second Finality. Technical Whitepaper. <https://somnia.network>.
- Stone, P. and Veloso, M. (2000). Multiagent systems: A survey from a machine learning perspective. *Autonomous Robots*, 8(3):345–383.
- Adams, H., Zinsmeister, N., Salem, M., Keefer, R., and Robinson, D. (2020). Uniswap v2 core. <https://uniswap.org/whitepaper.pdf>.
- Adams, H., Zinsmeister, N., Salem, M., Keefer, R., and Robinson, D. (2021). Uniswap v3 core. <https://uniswap.org/whitepaper-v3.pdf>.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., and Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. *NeurIPS 2022*.
- Wooldridge, M. (2009). *An Introduction to MultiAgent Systems*. John Wiley & Sons, 2nd edition.
- Xia, P., Wang, H., Zhang, B., Ji, R., Gao, B., Wu, L., Luo, X., and Xu, G. (2021). Trade or trick? detecting and characterising scam tokens on uniswap decentralised exchange. *ACM SIGMETRICS*.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. (2022). ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*.
- Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T. L., Cao, Y., and Narasimhan, K. (2023). Tree of thoughts: Deliberate problem solving with large language models. *arXiv preprint arXiv:2305.10601*.
- Cronje, A. (2020). Yearn Finance: Yield optimisation for stable coins. <https://yearn.finance>.